# InnoTech AI Healthcare System - Project Documentation

## 1. Project Planning

- **Project Overview:** A comprehensive end-to-end AI-powered healthcare platform (InnoTech) designed for automated Chest X-ray analysis, combining diagnostic CNN classification, U-Net lung segmentation, Grad-CAM interpretability, and a Retrieval-Augmented Generation (RAG) medical chatbot.
- **Dataset:** Indiana University Chest X-ray Collection (preprocessed to 3,727 clean records mapped between frontal images and clinical reports).
- **Project Phases & Architecture:**
  - **Phase 1 (Data Preprocessing):** Merging projections and reports, data cleaning, text preprocessing, image resizing, normalization, and generating structured splits (`rag_data.csv` and `cnn_data.csv`).
  - **Phase 2 (Core AI Models):**
    - *Phase 2A:* Multi-model CNN benchmarking (ResNet50, EfficientNet, DenseNet121) for binary classification (Normal vs. Abnormal) integrated with Grad-CAM heatmaps.
    - *Phase 2B:* U-Net lung mask extraction, medical text vectorization using LangChain and FAISS Vector Database.
  - **Phase 3 & 4 (Integration & Deployment):** FastAPI backend development (`/predict` and `/chat`), Android application UI via Retrofit API, and Docker containerization using Docker Compose.
- **Team Workflow & Parallel Tasks:** The work is distributed efficiently across parallel tracks: AI Preprocessing & CNN, U-Net Segmentation, RAG Chatbot Engine, FastAPI Backend, and Android Kotlin App.

## 2. Stakeholder Analysis :

- **Project Lead :** Responsible for architectural design, repository management, code review, merging Pull Requests, setting environment configurations (`config.py`), and ensuring smooth cross-functional team collaboration between local environments (VS Code) and cloud platforms (Google Drive / Colab).

- **AI / Deep Learning Engineers:** Responsible for training and benchmarking CNN models (ResNet50, EfficientNet, DenseNet121), implementing U-Net segmentation, and generating Grad-CAM heatmaps.
- **NLP & RAG Engineers:** Responsible for cleaning clinical reports, building embedding pipelines, and configuring LangChain with FAISS for the medical Q&A chatbot.
- **Backend & Mobile App Developers:** Responsible for building the FastAPI service, handling API endpoints, and developing the Android Kotlin interface with Retrofit integration.
- **End Users (Doctors & Medical Staff):** Primary beneficiaries who will utilize the mobile/web app to get instant X-ray classifications, visual explanations (Grad-CAM), and clinical insights.

# 3. Database Design

**Structured Data Layer (Processed CSVs):**

  - `rag_data.csv`: Stores textual reports, findings, and clinical impressions optimized for embedding generation and vector search.
  - `cnn_data.csv`: Maps image file names (`filename`), paths, and binary classification labels (`0` for Normal, `1` for Abnormal) along with clinical problem tags.
- **Vector Database (RAG):**
  - FAISS (Facebook AI Similarity Search) integrated with LangChain to store dense vector embeddings of medical literature and patient reports for fast semantic retrieval.
- **Image Data Management:**
  - Raw and normalized Chest X-ray images stored securely on shared Google Drive (`InnoTech_Dataset/images/images_normalized/`) with dynamic environment path detection (`config.py`) to bridge local VS Code and Google Colab environments seamlessly.

# 4. UI/UX Design

- **Platform Target:** Android Mobile Application built with Kotlin, designed for fast, accessible clinical use.
- **Key User Interfaces & Flows:**
  - **Authentication & Dashboard:** Secure login and home screen displaying system metrics and quick action buttons.

    - **X-Ray Upload & Analysis Screen:** Interface allowing medical staff to upload or capture a Chest X-ray image and instantly submit it to the backend.
    - **Diagnostic Results View:** Displays the binary classification result (Normal / Abnormal) accompanied by **Grad-CAM Heatmaps** to visually highlight regions of interest for the physician.
    - **AI Medical Assistant (Chatbot):** An interactive conversational interface powered by the RAG backend, enabling doctors to ask questions about symptoms, reports, or findings and receive context-aware medical responses.
- **Architecture & Communication:** Client-side Android app communicates with the Python-based FastAPI backend asynchronously via Retrofit API calls (`/predict` and `/chat` endpoints).